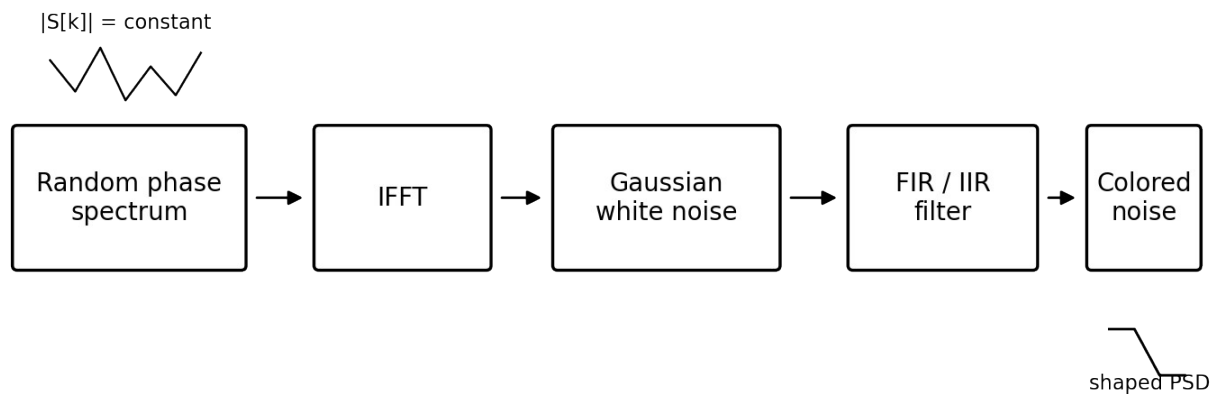


# Gaussian White and Coloured Noise Generation

Spectral synthesis, correlation analysis, FIR/IIR filtering, and frequency-domain masking



## Digital Signal Processing Laboratory

MATLAB implementation and technical report

**UPM**

Universidad Politécnica de Madrid

**Student: Guillermo Aladro**

Academic year: 2025/2026

Submission date: July 2026

# Contents

---

## *Numbered structure of the report*

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction and objectives</b>                 | <b>3</b>  |
| <b>2</b>  | <b>Theoretical basis</b>                           | <b>4</b>  |
| 2.1       | Random-phase spectral synthesis                    | 4         |
| 2.2       | Power spectral density and correlation             | 4         |
| <b>3</b>  | <b>MATLAB implementation: white-noise script</b>   | <b>5</b>  |
| <b>4</b>  | <b>White-noise results</b>                         | <b>6</b>  |
| <b>5</b>  | <b>Correlation analysis</b>                        | <b>7</b>  |
| <b>6</b>  | <b>FIR coloured-noise generation</b>               | <b>8</b>  |
| <b>7</b>  | <b>Coloured-noise implementation: IIR and mask</b> | <b>9</b>  |
| <b>8</b>  | <b>IIR and mask results</b>                        | <b>10</b> |
| <b>9</b>  | <b>Comparison and power verification</b>           | <b>11</b> |
| <b>10</b> | <b>Correlation discussion and conclusions</b>      | <b>12</b> |
| <b>A</b>  | <b>Execution guide and file structure</b>          | <b>13</b> |

## **Deliverables**

- English technical report in PDF format.
- Two executable MATLAB scripts: RuidoBlancoGaussiano.m and RuidoColoreado.m.
- Representative output figures and a concise README file.

# 1 Introduction and objectives

---

*Scope of the supplied MATLAB practice*

The supplied code implements stochastic signal synthesis rather than a deterministic recursive oscillator. It constructs Gaussian white noise from a constant-magnitude spectrum with random phase, analyses its spectrum and correlations, and then creates coloured noise using FIR filtering, Butterworth IIR filtering, and an ideal frequency-domain mask.

## **Central engineering idea**

A flat spectral magnitude with independent random phases produces a time-domain sequence whose real and imaginary components are approximately Gaussian. Shaping the spectrum or filtering the sequence introduces temporal correlation and therefore creates coloured noise.

The main objectives are:

- Generate a reproducible complex random process with a prescribed total power.
- Extract statistically equivalent real and imaginary white-noise components.
- Estimate and interpret a one-sided power spectral density (PSD).
- Use auto-correlation and cross-correlation to assess temporal memory and component independence.
- Compare finite impulse response (FIR), infinite impulse response (IIR), and direct spectral-mask colouring methods.
- Verify simulated power against the theoretical relationship  $P = N_0 B$ .

The original algorithms were retained, while the delivered scripts correct the one-sided PSD scaling, use explicit frequency vectors, create the mask from physical band limits, and employ monochrome plots that remain legible when printed.

## 2 Theoretical basis

---

*Noise synthesis, spectral estimates, and correlation*

### 2.1 Random-phase spectral synthesis

For an N-point discrete spectrum, every frequency bin is assigned the same magnitude K and an independent phase uniformly distributed in  $[0, 2\pi)$ . The spectrum is therefore

$$S[k] = K \exp(j 2 \pi u[k]), \quad u[k] \sim U(0,1)$$

The inverse FFT adds many phasors at each time sample. By the central limit effect, the real and imaginary parts approach Gaussian distributions. Multiplication by  $\sqrt{2}$  compensates for splitting the complex-process power between the two quadratures.

### 2.2 Power normalisation and PSD

For a one-sided white-noise density  $N_0$  over the interval 0 to  $f_s/2$ , the expected noise power is  $P_n = N_0 f_s/2$ . With reference impedance  $Z$  and the FFT/IFFT normalisation used by MATLAB, the constant spectral magnitude is selected as

$$K = \sqrt{P_n Z N}$$

The scripts compute  $X = \text{fft}(x)/N$  and  $P_{xx} = |X|^2/Z$ . A one-sided estimate retains bins 0 to  $f_s/2$  and doubles all strictly positive bins, because the removed negative-frequency half contains equal power for a real sequence.

### 2.3 Correlation interpretation

Ideal white noise has an impulse-like autocorrelation: only zero lag is significant. Filtering spreads the autocorrelation over several lags because adjacent samples share common past inputs. The cross-correlation between the real and imaginary components should remain small, which indicates that the two quadratures behave as independent noise realisations.

### 3 MATLAB implementation: white-noise script

*RuidoBlancoGaussiano.m*

The first script is organised into six blocks: parameter definition, random-phase synthesis, one-sided PSD calculation, white-noise correlation analysis, FIR colouring, and numerical power checks.

| Parameter | Value     | Meaning                          |
|-----------|-----------|----------------------------------|
| N         | 1024      | Number of samples and FFT length |
| fs        | 1024 Hz   | Sampling frequency               |
| Z         | 50 ohm    | Reference impedance              |
| N0        | 1e-6 W/Hz | One-sided white-noise PSD        |
| Pn        | 5.12e-4 W | Expected total white-noise power |
| fc        | 128 Hz    | FIR low-pass cut-off frequency   |
| FIR order | 30        | Hamming-window filter order      |

#### Core synthesis block

```
Pn = N0*(fs/2);
K = sqrt(Pn*Z*N);
S = K*exp(1j*2*pi*rand(1,N));
x = ifft(S);
x1 = sqrt(2)*real(x);
x2 = sqrt(2)*imag(x);
```

The phase-only randomisation keeps the energy per DFT bin fixed while changing the instantaneous waveform. Because the same complex sequence supplies both quadratures, x1 and x2 have the same statistical power but independent-looking sample patterns.

#### One-sided PSD correction

```
PxxOne = Pxx(1:N/2);
PxxOne(2:end) = 2*PxxOne(2:end);
f = (0:N/2-1)*(fs/N);
```

The frequency vector explicitly uses the FFT-bin spacing  $fs/N$ . This avoids relying on a unit-step colon expression and makes the script correct if N and fs are changed independently.

## 4 White-noise results

### Time-domain appearance and spectral flatness

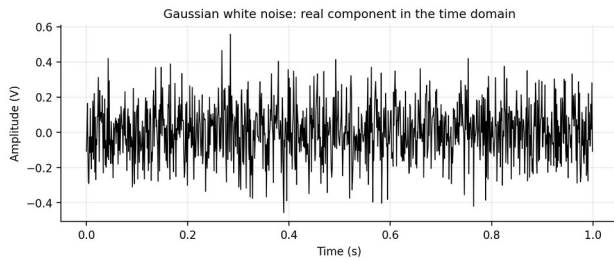


Figure 1. One realisation of the real Gaussian component.

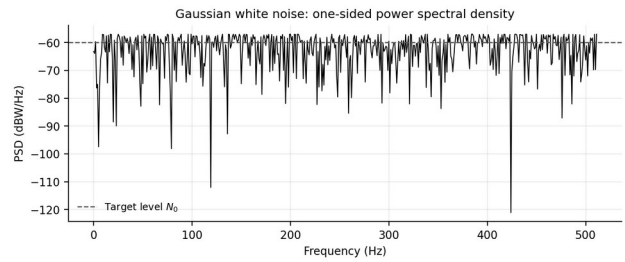


Figure 2. One-sided PSD with the target  $N_0$  reference level.

The time-domain waveform is irregular, centred near zero, and contains rapid sample-to-sample changes. No periodic structure is visible. These properties are expected from a broadband random process.

The periodogram fluctuates strongly around the target density because it is based on only one finite record. This variance is not an implementation error: an unsmoothed periodogram is an inconsistent estimator, so increasing  $N$  improves frequency resolution but does not remove all local fluctuations. The important result is that no frequency region is systematically favoured before filtering.

**Representative power check:** expected  $5.1200\text{e-}4$  W; estimated  $4.9196\text{e-}4$  W (approximately -3.9% error for this finite realisation).

The sample estimate converges statistically to the theoretical power. A different seed produces a different waveform and a slightly different finite-record estimate while preserving the same expected value.

## 5 Correlation analysis

### Whiteness and quadrature independence

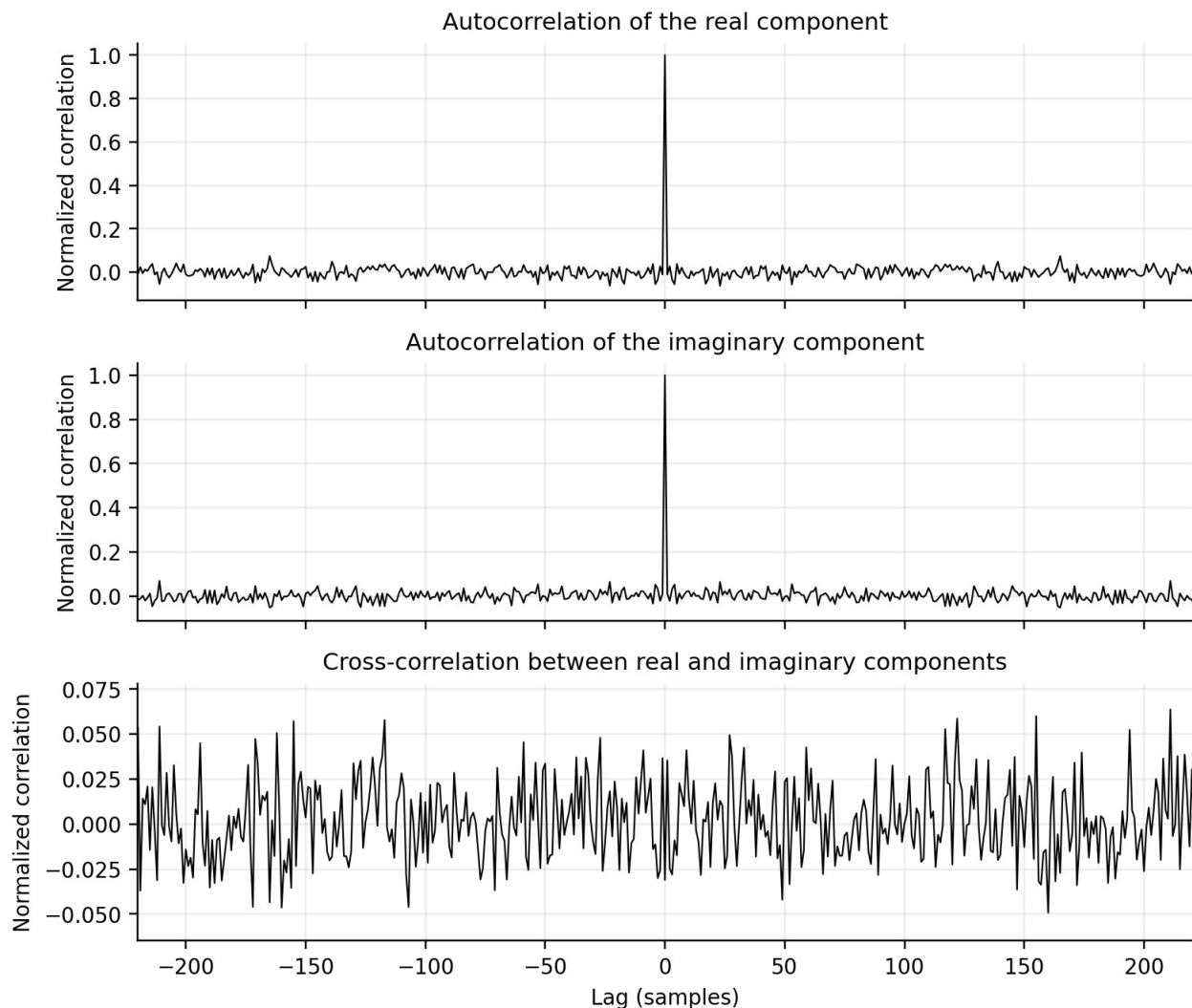


Figure 3. Normalised auto-correlations and cross-correlation of the white-noise components.

Both autocorrelation curves contain a dominant zero-lag peak and small values elsewhere. The non-zero-lag ripples are finite-sample effects: a record of 1024 points cannot reproduce an exact Kronecker delta.

The real-imaginary cross-correlation remains close to zero and has no stable central peak. This supports the interpretation of the two quadratures as separate noise channels with negligible linear dependence. In the MATLAB scripts, `xcorr` is intentionally left unnormalised so that absolute correlation energy is available; the report figure is normalised only to simplify visual comparison.

## 6 FIR coloured-noise generation

*Low-pass spectral shaping and temporal memory*

### FIR colouring stage

```
Wn = fc/(fs/2);  
filterOrder = 30;  
b = fir1(filterOrder, Wn, 'low');  
y1 = filter(b, 1, x1);  
y2 = filter(b, 1, x2);
```

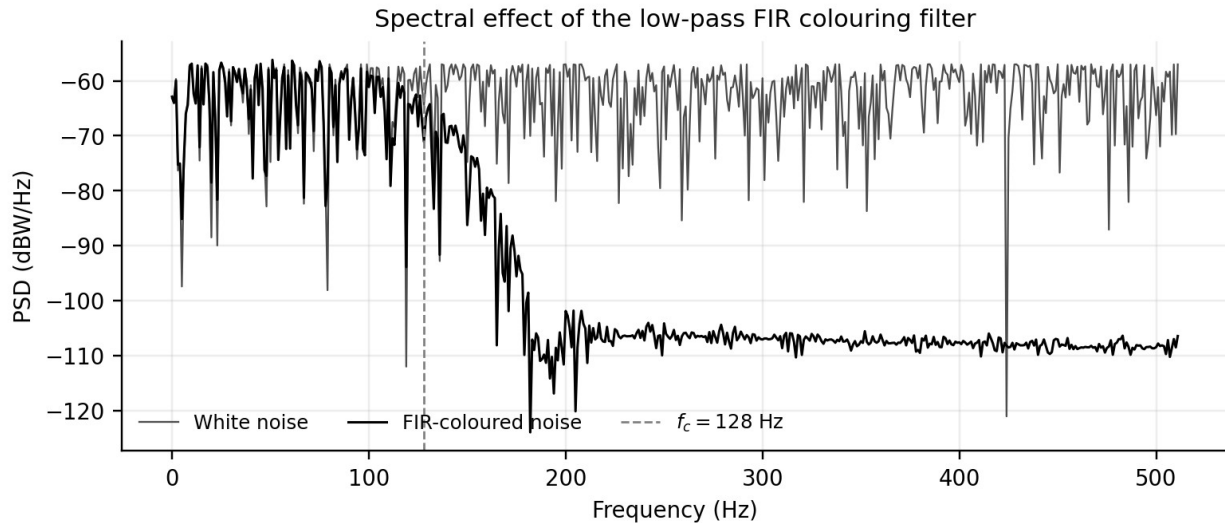


Figure 4. White-noise PSD and the low-pass FIR-coloured output.

The order-30 Hamming-window FIR filter preserves the low-frequency region and attenuates components above approximately 128 Hz. The transition band is finite because a realizable FIR filter cannot implement an ideal discontinuity with a limited number of coefficients.

After filtering, the process is no longer white. Its spectral power depends on frequency, and its samples become correlated over a duration comparable to the impulse-response length. The representative estimated output power is  $1.0600 \times 10^{-4}$  W, below the ideal rectangular-band value of  $1.2800 \times 10^{-4}$  W because the real FIR response includes a transition band and attenuation near the cut-off, and because the initial filter transient is included in the finite record.



## 7 Coloured-noise implementation: IIR and mask

*RuidoColoreado.m*

The second script compares two methods for selecting the normalised band  $[1/3, 2/3]$ , which corresponds to the physical interval  $f_s/6$  to  $f_s/3$ . With  $f_s = 512$  Hz, the desired band is 85.333 Hz to 170.667 Hz.

| Method          | Implementation                              | Main property                                  |
|-----------------|---------------------------------------------|------------------------------------------------|
| Butterworth IIR | <code>butter(4,[1/3 2/3],'bandpass')</code> | Smooth passband and finite transition regions  |
| Ideal mask      | <code>Scol = S .* mask</code>               | Exact FFT-bin selection with abrupt band edges |

### Frequency-domain mask

```
fmin = (1/3)*(fs/2);
fmax = (2/3)*(fs/2);
fTwoSided = (0:N-1)*(fs/N);
mask = ((fTwoSided >= fmin) & (fTwoSided <= fmax)) | ...
       ((fTwoSided >= fs-fmax) & (fTwoSided <= fs-fmin));
Scol = S.*mask;
yMask = ifft(Scol);
```

The second logical interval is the negative-frequency mirror of the positive passband. Keeping both halves is essential: removing the mirror would produce an asymmetric complex spectrum and change the statistics of the real and imaginary outputs.

The IIR method is causal and efficient for streaming implementation, but it has nonlinear phase and an internal transient. The mask is non-causal as an offline block operation and corresponds to a very long periodic impulse response, but it produces a sharply defined spectrum on the FFT grid.

## 8 IIR and mask results

### *Time-domain behaviour and selected bandwidth*

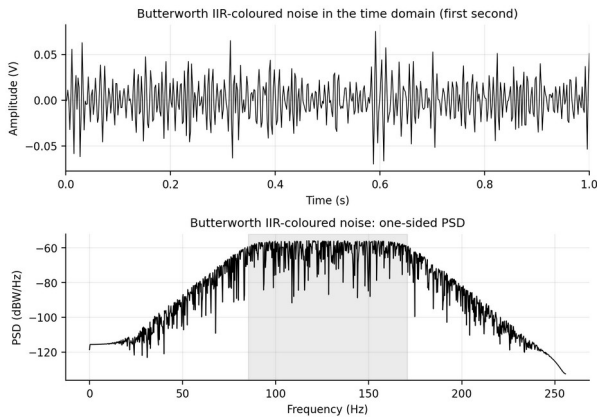


Figure 5. Fourth-order Butterworth band-pass output.

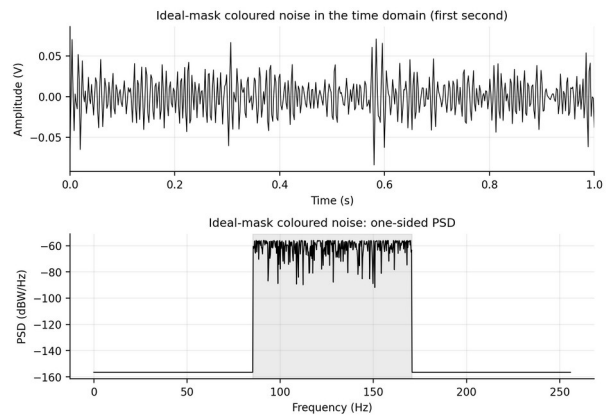


Figure 6. Output obtained with the ideal FFT-bin mask.

Both methods confine most of the energy to the requested band. The Butterworth response changes gradually around the edges, whereas the mask switches between rejected and retained bins without a transition region.

The band-limited waveforms appear smoother than white noise because the highest and lowest frequency components have been removed. They also show oscillatory bursts whose typical time scale is related to the centre frequency of the passband. This oscillatory appearance does not imply a deterministic sinusoid; the amplitude and phase remain random.

The ideal-mask signal is produced by a block FFT operation, so its first and last samples are implicitly connected under the DFT periodicity assumption. The IIR output, by contrast, is generated sequentially and includes a start-up transient from zero initial conditions.

## 9 Comparison and power verification

*Theoretical bandwidth power versus simulation*

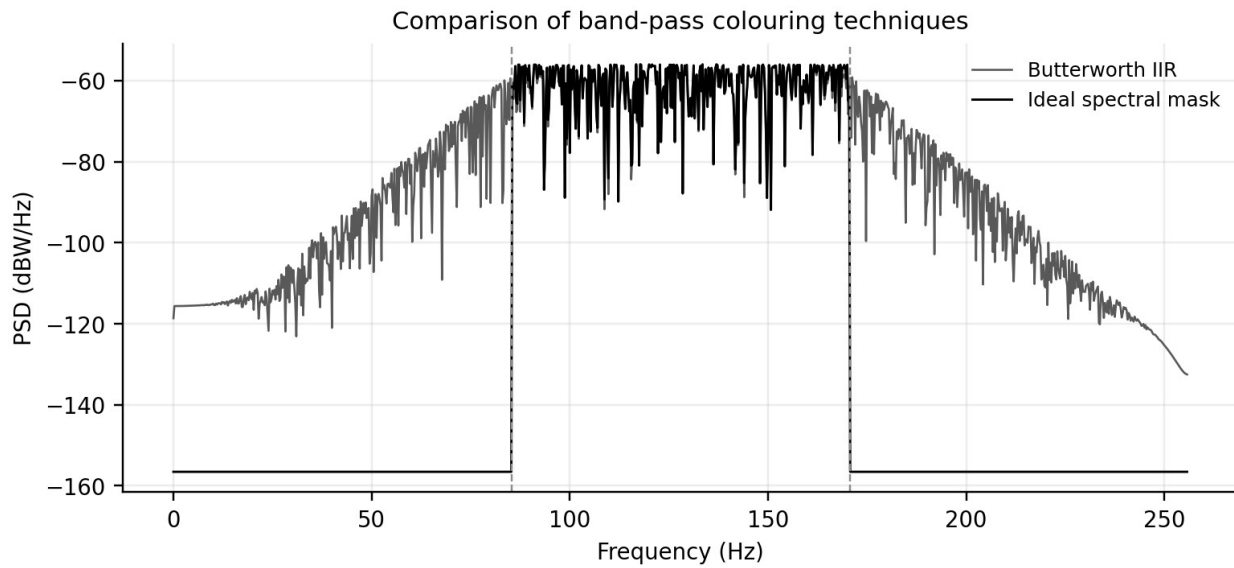


Figure 7. Spectral comparison of the Butterworth and ideal-mask methods.

| Quantity             | Theoretical / expected   | Representative estimate |
|----------------------|--------------------------|-------------------------|
| White-noise power    | 5.1200e-4 W              | 4.9196e-4 W             |
| FIR low-pass output  | 1.2800e-4 W (ideal band) | 1.0600e-4 W             |
| IIR band-pass output | 4.2667e-4 W (ideal band) | 4.3771e-4 W             |
| Ideal-mask output    | 4.2667e-4 W              | 4.3812e-4 W             |

For the second script,  $B = f_s/3 - f_s/6 = 85.333$  Hz and therefore  $P_{\text{expected}} = N_0 B = 4.2667\text{e-}4$  W. The ideal-mask estimate is close to this value; the remaining difference comes from the discrete inclusion of edge bins and the random distribution of real-component power in a finite record.

The Butterworth estimate can be slightly above or below the rectangular-band result because its magnitude response is not a perfect box. Some energy is attenuated inside the nominal band, while a smaller amount passes through the transition regions outside it. The exact finite-record value also depends on the transient and random seed.

## 10 Correlation discussion and conclusions

*What spectral colouring changes in the time domain*

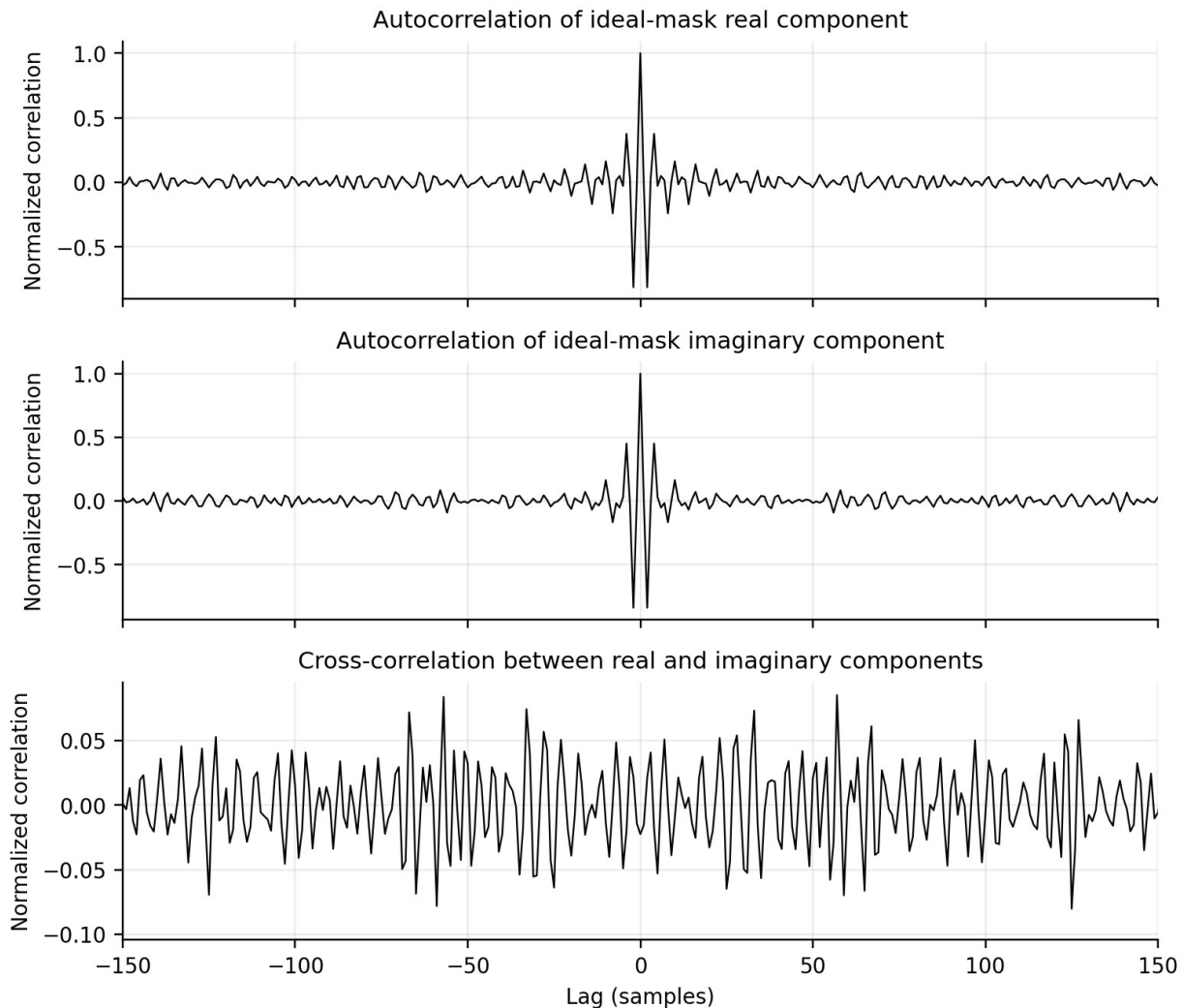


Figure 8. Correlation functions of the ideal-mask coloured process.

Unlike white noise, the band-pass process has an oscillatory autocorrelation extending over many lags. This is the time-domain counterpart of its restricted spectrum: a narrow or structured frequency response necessarily introduces memory. The envelope decays because the passband has non-zero width rather than containing a single spectral line.

The cross-correlation between the real and imaginary components remains much smaller than the zero-lag autocorrelation. Filtering both components with the same linear system changes their individual autocorrelations but does not create a strong deterministic dependence between them.

### Conclusions

- Random-phase spectral synthesis is an efficient way to obtain a finite Gaussian-like noise record with controlled expected power.
- Correct one-sided PSD scaling is necessary for meaningful comparison with the specified density  $N_0$ .
- White noise is identified by an approximately flat PSD and impulse-like autocorrelation.
- FIR and IIR filters create realizable coloured processes with transition bands and implementation transients.
- A spectral mask provides precise FFT-bin selection but is naturally an offline block technique.
- The simulated power values agree with  $N_0 B$  within the deviations expected from finite data, filter shape, and bin discretisation.

# A Execution guide and file structure

---

*Reproducing the practice in MATLAB*

The ZIP archive uses the following structure:

```
Practica_Ruido_Blanco_Coloreado/  
|-- Report_Gaussian_White_and_Coloured_Noise.pdf  
|-- README.txt  
|-- MATLAB/  
|   |-- RuidoBlancoGaussiano.m  
|   |-- RuidoColoreado.m  
|-- Results/  
|   |-- 01_white_noise_time.png  
|   |-- ...  
|   |-- 10_mask_coloured_correlations.png
```

Run the scripts from the MATLAB folder:

```
cd MATLAB  
RuidoBlancoGaussiano  
RuidoColoreado
```

MATLAB with Signal Processing Toolbox is required because the scripts call `fir1`, `butter`, and `xcorr`. The random generators are initialised with fixed seeds so that results are reproducible. Replace `rng(11,'twister')` or `rng(23,'twister')` with `rng('shuffle')` when a new realisation is desired on every execution.

## Quality and implementation notes

- Figures in this report were reproduced numerically with the same equations and parameters as the delivered MATLAB scripts.
- The scripts plot linear PSD values, while selected report plots use dBW/Hz to make attenuation and band edges easier to inspect.
- No long lookup table is required: the noise sequence is generated from an FFT-domain description and filtered algorithmically.
- Changing `N` modifies record length and frequency resolution; changing `fs` modifies the physical interpretation of every frequency bin.

End of report.